

# 2IMP20 Domain Specific Language Design Assignment 3 – Group 27

Johan van Dongen  
j.p.v.dongen@student.tue.nl  
Jeroen Oerlemans  
j.r.oerlemans@student.tue.nl

## 1 Introduction

### 1.1 Motivation

Creating visual animations and illustrations, especially for educational, generative, or simulation purposes, often requires either significant programming experience or manual effort. Tools like Manim offer expressive control but can be overwhelming for users without a computer science background. On the other end of the spectrum, tools like Photoshop or Illustrator are widely used for visual design but are limited when it comes to dynamic behavior, or procedural pattern generation.

Our goal is to bridge this gap by designing a domain-specific language (DSL) that enables users to easily create animated SVG graphics using simple, declarative syntax. Unlike general-purpose tools, our DSL allows users to define rules, shapes, and behaviors programmatically, making it particularly well-suited for simulations, repetitive layouts, or algorithmic visual effects.

By abstracting away boilerplate code and providing high-level constructs tailored to 2D graphics, our DSL lowers the barrier to entry while still supporting complex visual output. This empowers designers, educators, and students, especially those without deep programming experience, to generate dynamic visuals in an intuitive and modular way.

### 1.2 Forma Language Description

Forma is a language used to specify JSON like objects to convert to simple webpages. A user can specify a easy-to-interpret file and the Forma language will take the values specified by the user to generate an animated, custom visualization in the form of a JavaScript supported HTML webpage. The user can specify a variety of shapes and perform multiple actions on those defined shapes. Such actions feature looping multiple instances of a shape, copying shapes, drawing them and defining animating specific conditional and unconditional rules.

### 1.3 Forma Language Example

Below, an example Forma file is given that explores all the features of the Forma language:

```
1 Circle c1 {  
2   color = random,
```

```

3     radius = random,
4     pos = random
5 };
6
7 Rectangle r1 {
8     color = random,
9     width = 50,
10    height = 50,
11    pos = random
12 };
13
14 Triangle t1 {
15     color = random,
16     width = 50,
17     height = 50,
18     pos = random
19 };
20
21 Repeat houses {
22     shape = t1,
23     amount = 20
24 };
25
26 Copy r1 as [r2];
27 Copy houses as [villas, mansions];
28
29 draw r2;
30 draw c1;
31 draw villas;
32
33 Rule t1.width += 1;
34 Rule r1.height += 1;
35 Rule c1.color = "blue" when c1.radius > 50;

```

Listing 1: Example Forma file

In the Syntax segment, we will more closely zoom in on the different aspects of this forma file. It can be seen that the user specifies three base shapes, Circle c1, Rectangle r1 and Triangle t1 each with their own set properties.

The user then creates a variable loop of triangles called houses. This loop functionality contains 20 units of t1 all keeping their t1 object identity (important for rules). The user then copies r1 once to create a separate Rectangle instance r2 and does the same for houses. Villas and Mansions thus share t1 rule functionality where r2 does not share r1 rule functionality.

The user draws the rectangle r1, circle c2 and the loop of triangles "villas". The rules specified alter the specifics on these objects, the first rule increasing t1.width applies to all triangles in villas. The r1 rulings are not shown, as r1 is not drawn. The conditional rule then updates the color of drawn circle c1 when its radius exceeds 50.

## 2 Concrete Syntax

Concrete Forma syntax functions in a way where top-level elements are constructed with identifiers to be used elsewhere in the syntax to either perform actions on, or reuse for different purposes. The Forma syntax consists of different statements to identify what code should be generated in the future. The Top-Level Syntax looks as follows:

```

start syntax Forma = forma: {Statement ";" }+ stmts ";";
syntax Statement
| = circle: "Circle" Ident name Properties
| rectangle: "Rectangle" Ident name Properties
| triangle: "Triangle" Ident name Properties
| draw: "draw" Ident name
| repeat: "Repeat" Ident name "{" RepeatFields "}"
| rule: "Rule" Target Operator Expression "when" Condition
| ruleUni: "Rule" Target Operator Expression
| copy: "Copy" Ident name "as" "[" {Ident ","}* "]" ;

```

Figure 1: Top-Level Syntax

Figure 1 clearly shows the concrete Syntax of the Forma language. This syntax consists of three shapes and different operations that can be executed on defined shapes. Below, we will highlight each of these features in detail

## 2.1 Shapes

Forma features three different shape statements: Circles, Rectangles and Triangles. Shapes are identified by an identifying name and have a list of properties. Take the forma specification below:

```

Triangle t1 {
    color = "red",
    width = 60,
    height = 40,
    pos = random
};

```

This triangle shape shows the implementation of this shape Syntax. It has a shape string "Triangle", an identifying name "t1" and various properties.

### 2.1.1 Properties

Shape properties are what allows a user to define specifics within a shape. The property syntax is defined as such:

As is seen in Figure 2, properties can be set based on the type of shape. The well-formedness of the shapes is as follows:

#### Circles:

- color - String value or random
- radius - Integer value or random
- position - Tuple of (x,y) or random

#### Rectangles and Triangles:

- color - String value or random

```

syntax Pos = pos: "(" Number x "," Number y ")" | random: "random";
syntax NumberValue = val: Number | random: "random";
syntax StringValue = val: String | random: "random";

syntax Properties = "{" {Property ","}* "}";
syntax Property
  = color: "color" "=" StringValue
  | radiusProp: "radius" "=" NumberValue
  | widthProp: "width" "=" Number width
  | heightProp: "height" "=" Number height
  | posProp: "pos" "=" Pos;

```

Figure 2: Property Syntax

- width - Integer value
- height - Integer value
- position - Tuple of integers (x,y) or random

Random values are in the syntax a simple string "random" and will be handled by the Code Generation. We can clearly see that Circles have a different well-formedness property. However, circles can be initialized with a width and height and rectangles can be initialized with a radius, but the Code Generation will not implement these specifications.

## 2.2 Drawing

The Syntax of drawing is quite simple. It has no arguments other than an identifying name. The intention of draw is that shapes (or repeats of shapes) that should not be displayed remain hidden. Only once an object is called by draw will the object be displayed by JavaScript.

## 2.3 Repeat

The repeat syntax allows a user to specify objects to be repeated a number of times. The specification in the forma files looks like this:

```

Triangle t1 {
  color = "red",
  width = 60,
  height = 40,
  pos = random
};

Repeat pyramids {
  shape = t1,
  amount = 2
};

```

Included is the specification for the Triangle that is repeated. The Syntax that's based on this forma implementation is seen below

A repeat is simply done by specifying a shape string of an existing shape in a user's forma file and the amount of loops that should occur.

```
data RepeatField
  = shapeField(str shape)
  | amountField(str amount);
```

## 2.4 Rules

Rules are, as mentioned before, what allows a user to specify operations on the shapes during runtime. An example of a rule can be seen below:

```
Rule c1.radius += 1;
Rule c1.color = "blue" when c1.radius > 50;
```

Rules have a rather complex structure with various elements, below a more isolated overview of the Rule syntax can be seen:

```
rule: "Rule" Target Operator Expression "when" Condition
ruleUni: "Rule" Target Operator Expression
```

Rules are split into two parts: Regular rules and Universal rules. Regular rules have a conditional aspect where universal rules hold universally, without a conditional aspect. This distinction allows for a smoother generation of JavaScript by filtering if there's a condition to the rule in question. Now we will break down the rule syntax further. Every rule has three or four elements depending on the type of rule:

- Target - The target object in question. In our example above this would be `c1.radius` or `c1.color`
- Operator - A JavaScript representation of operators. In our example these are `+=` or `=`
- Expression - The value set to the Target. In our example `1` or `"blue"`
- (Optional) Condition - The entire statement after `"when"`. In our example `"c1.radius > 50"`. This has to match a JavaScript expression.

The syntax for these rules is as follows:

We can clearly see the semantics. A target is defined as a point-separated object (in our example `c1` and `radius` or `c1` and `color`). Furthermore, we see that in that Syntax level of the potential condition, we follow a similar structure, with a target, a relational operator (following JavaScript syntax) and an expression.

## 2.5 Copy

Copying is similar to the repeat function, but rather than blindly repeating a shape, we can use copy to store the repeats into their own object names. Take the form specification below:

```
Circle c1 {
  color = random,
  radius = random,
  pos = random
};
```

```
syntax Condition = condition: Target RelOp Expression;
```

```
syntax Target = target2: Ident "." Ident;
```

```
syntax Operator
```

```
  = eq: "="  
  | addEq: "+="   
  | subEq: "-=";
```

```
syntax Expression
```

```
  = strExpr: String  
  | random: "random"  
  | posExpr: Pos  
  | intExpr: Number  
  | refExpr: Ident;
```

```
syntax RelOp
```

```
  = gt: ">"  
  | lt: "<"  
  | ge: ">="   
  | le: "<="   
  | eq: "=="  
  | ne: "!=";
```

```
Triangle t1 {  
  color = random,  
  width = 50,  
  height = 50,  
  pos = random  
};
```

```
Repeat houses {  
  shape = t1,  
  amount = 20  
};
```

```
Copy c1 as [c2];  
Copy houses as [villas, mansions];
```

This example shows a clear representation of the Copy functionality. Circle c2 is instantiated using c1, copying can even be done in mass using the specification seen on houses. Repeats can also be copied using this specification. The Syntax for copying is the same as the Statement level specification:

```
copy: "Copy" Ident name "as" "[" {Ident ","}* "]"
```

Clearly showing the forma specification syntax described above.

## 2.6 Abstract Syntax

To convert the concrete Syntax to AST, we run the Rascal given `#implode` function. This creates an Abstract syntax representation of the forma file specified. To ensure this process matches smoothly, an almost one-to-one Abstract specification is created. The Abstract Syntax specification can be seen below:

```
1 data Forma = forma(list[Statement] stmts);
2
3 data Statement
4   = circle(str name, list[Property] props)
5   | rectangle(str name, list[Property] props)
6   | triangle(str name, list[Property] props)
7   | draw(str name)
8   | repeat(str name, list[RepeatField] fields)
9   | rule(Target target, str op, Expression exp, Condition condition)
10  | ruleUni(Target target, str op, Expression exp)
11  | copy(str from, list[str] to);
12
13 data Pos
14   = pos(str x, str y)
15   | random();
16
17 data Property
18   = color(str val)
19   | radiusProp(str val)
20   | widthProp(str w)
21   | heightProp(str h)
22   | posProp(Pos pos);
23
24 data RepeatField
25   = shapeField(str shape)
26   | amountField(str amount);
27
28 data Target = target2(str base, str field);
29
30 data Expression
31   = strExpr(str val)
32   | intExpr(str val)
33   | refExpr(str name)
34   | posExpr(Pos pos)
35   | random();
36
37 data Condition = condition(Target left, str op, Expression right);
```

Listing 2: AST Specification

With this Abstract Syntax implementation, the `implode` function casts a forma file compliant to the Syntax specified earlier into an Abstract Syntax seen in Listing 2 and thus creates the AST to be used by Code Generation.

## 3 Code Generation

Our DSL compiles into a web-based project. Each `.frm` file is translated into two output files: an `.html` file and a corresponding `.js` file. The HTML file is minimal and contains standard boilerplate code along with an SVG `viewBox` element, which acts as the drawing canvas. It also includes a `<script>` tag that links to the generated JavaScript file. An example HTML output can be seen in Listing 9.

The JavaScript file handles all dynamic behavior. It selects the SVG canvas and uses standard DOM operations to draw shapes by appending newly created SVG elements. Animations are implemented using vanilla JavaScript and the `requestAnimationFrame` API. This forms an infinite animation loop, within which we update the attributes of the previously created SVG elements. Listing 3 shows the JavaScript initialization and listing 4 shows the logic for a basic animation loop without any rules applied yet.

```
1 const svg = document.getElementById("canvas");
2 const svgNS = "http://www.w3.org/2000/svg";
3 svg.setAttribute("viewBox", "0 0 500 500");
```

Listing 3: JS Initialization

```
1 fps = 60;
2 fpsInterval = 1000 / fps;
3 then = window.performance.now();
4 startTime = then;
5 function animate() {
6     requestAnimationFrame(animate);
7     now = window.performance.now();
8     elapsed = now - then;
9     if (elapsed > fpsInterval) {
10         then = now - (elapsed % fpsInterval);
11         // Animation updates
12         // Rules
13         // Unconditional rules
14     }
15 }
16 }
17 animate();
```

Listing 4: JS Animation loop

### 3.1 Shapes

For every shape encountered in the AST, we generate a corresponding JavaScript class, which we refer to as a *shape generator*, or simply a *generator*. Each generator’s constructor initializes the SVG element and sets its attributes according to the DSL definition. Additionally, each generator includes getter and setter methods for all defined shape properties. A global array is also created for each shape type to store all instances of that shape. See Listing 5 for an example transformation.

The decision to create a dedicated class for each shape definition, referred to as a generator, enables shapes to be instantiated multiple times in a consistent and modular way. This design choice aligns well with the underlying principles of our DSL. Moreover, the class-based structure simplifies animation logic by encapsulating both the shape’s data and its behavior. Animation will be discussed in more detail in Section 3.2.

```
1 // .frm
2 Circle circle {
3     color = "blue",
4     radius = random,
```



```

5 };
6
7 // .js
8 // Simplified class without setters/getters
9 class GeneratorCircle {
10     constructor() {
11         this.svg = document.createElementNS(svgNS, "circle")
12         svg.appendChild(this.svg)
13         this.setcolor("blue")
14         this.setradius("random")
15     }
16 }
17 circle = [];

```

Listing 5: Shape definition transformation

An example of the encapsulation of a property is shown in Listing 6. Each property includes a setter that updates the internal value (in this case, `radius`) and synchronizes the corresponding SVG attribute. The getter returns the current value of the property, allowing it to be used in rules and conditions.

Setters may also invoke utility functions such as `getRandomSmall`, which are automatically generated and included at the top of the JavaScript output, regardless of the contents of the `.frm` file. Another example of such auto-generated support is the `Pos` class, which simplifies position-related operations.

```

1 // .js
2 setradius(radius) {
3     this.radius = radius === "random" ? getRandomSmall() : radius
4     this.svg.setAttribute("r", this.radius)
5 }
6 getradius() {
7     return this.radius;
8 }

```

Listing 6: getter/setter transformation

Whenever the `draw` keyword is encountered in the AST, a new shape instance is created and stored in the corresponding list. Instantiating a shape automatically appends it to the SVG canvas.

```

1 // .frm
2 draw circle
3 draw circle
4
5 // .js
6 circle.push(new GeneratorCircle())
7 circle.push(new GeneratorCircle())

```

Listing 7: Draw transformation

## 3.2 Animation

Rules defined in the DSL are translated into update statements that are executed within the animation loop. Thanks to the generated getter and setter methods, this transformation is straightforward.

Each rule specifies which property of a shape should be updated. Accordingly, we call the appropriate setter method for every instance created by the corresponding shape generator.

If the rule involves modifying the property based on its current value (e.g., `+=`, `-=`), we first retrieve the existing value using the getter, apply the expression, and pass the result to the setter. In contrast, for assignments involving a completely new value (e.g., `=`), we directly pass the value to the setter without referencing the current state.

For evaluating conditions in conditional rules, we simply use the corresponding getter methods to access the current properties of each instance. See listing 8 for an example transformation.

```
1 // .frm
2 Rule circle.radius += 1;
3 Rule circle.color = "blue" when c1.radius > 50;
4
5 // .js
6 // The snippet below is placed in the animation loop (note the matching comments)
7
8 // Rules
9 for (i=0; i < circle.length; i++) {
10     circle[i].setradius(circle[i].getradius() + 1);;
11 }
12
13 // Conditional rules
14 for (i=0; i < circle.length; i++) {
15     if (circle[i].getradius() > 50) {
16         circle[i].setcolor("blue");
17     }
18 }
```

Listing 8: Rule definition transformation

```
1
2 <!DOCTYPE html>
3 <html lang="en" >
4     <head>
5         <meta charset="UTF-8">
6         <title>Forma</title>
7         <style>
8             * {
9                 margin: 0;
10                padding: 0;
11                box-sizing: border-box;
12            }
13            html, body {
14                background: #f0f0f0;
15                height: 100vh;
16                width: 100vw;
17                display: flex;
18                justify-content: center;
19                align-items: center;
20            }
21        </style>
22    </head>
23    <body>
24    </body>
25 </html>
```

```

21
22     svg {
23         max-width: 100%;
24         max-height: 100%;
25         border: 0.2rem solid black;
26         border-radius: 1rem;
27         background: white;
28     }
29 </style>
30 </head>
31 <body>
32     <svg
33         id="canvas"
34         viewBox="0 0 100 100"
35         preserveAspectRatio="xMidYMid meet"
36         xmlns="http://www.w3.org/2000/svg"
37     ></svg>
38     <script src="circle.js" ></script>
39 </body>
40 </html>

```

Listing 9: HTML Example

## 4 Resulting Webpages

Using our DSL, we can generate complete web-based projects that feature visually appealing SVG images and animations using simple syntax. Figure 3 displays the set of basic shapes supported by the language: Circle, Rectangle, and Triangle. Figure 4 illustrates a more complex output: a dynamic composition of multiple circles generated using the repeat statement, each assigned randomized attributes such as position, radius, and color. While animations cannot be rendered directly within this static report, the resulting visuals demonstrate the expressive power and creative flexibility of our language.



Figure 3: Supported shapes

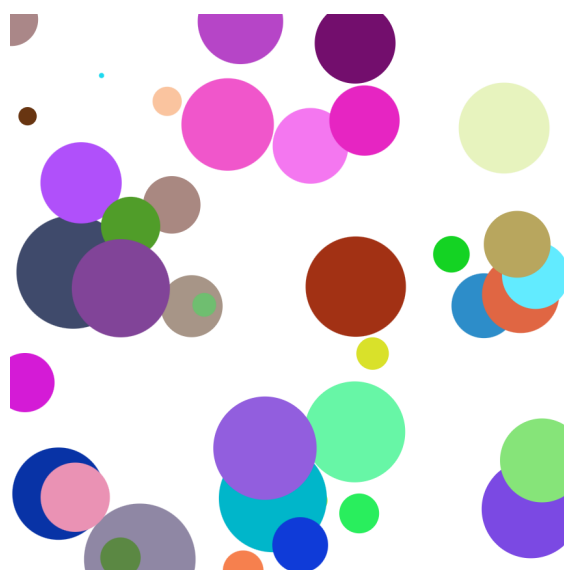


Figure 4: Slash composition

## References

- [1] Paul Klint, Tijs van der Storm, and Jurgen J. Vinju. *Easy meta-programming with Rascal: Leveraging the Extract-Analyze-Synthesize paradigm for meta-programming*. In *Proceedings of the 3rd International Summer School on Generative and Transformational Techniques in Software Engineering (GTTSE'09)*, LNCS, Springer, 2010.
- [2] Grant Sanderson et al. *Manim: Mathematical Animation Engine*. Available at: <https://github.com/ManimCommunity/manim>. Accessed July 2025.
- [3] W3C. *Scalable Vector Graphics (SVG) 1.1 (Second Edition)*. W3C Recommendation, 2011. Available at: <https://www.w3.org/TR/SVG11/>
- [4] ECMA International. *ECMAScript® Language Specification (ECMA-262)*. 11th edition, 2020. Available at: <https://www.ecma-international.org/publications-and-standards/standards/ecma-262/>